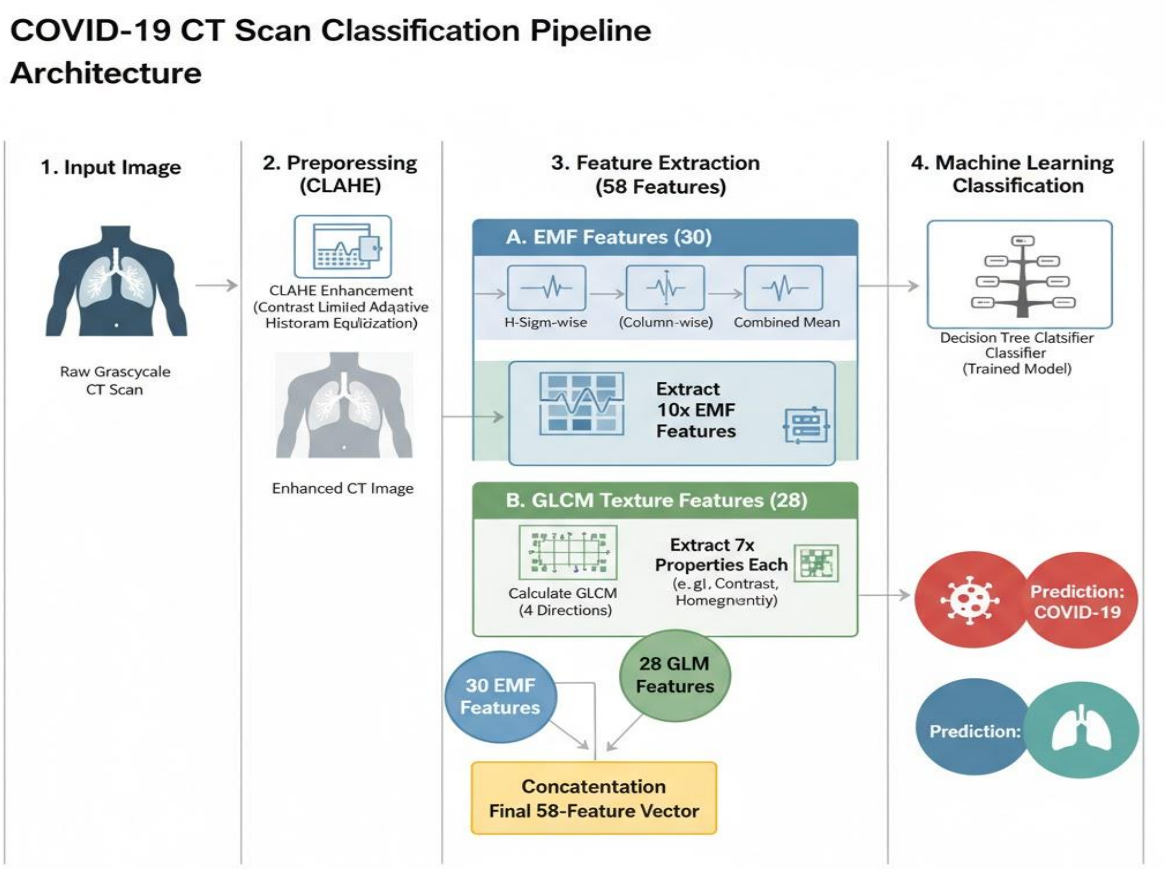
COVID-19 Detection Using Electromagnetic (EM) Feature Extraction and Machine Learning




```
import numpy as np
import pandas as pd
import os

base_path = "/kaggle/input/covid19-lung-ct-scans/COVID-19_Lung_CT_Scans/"
categories = ["COVID-19", "Non-COVID-19"]

image_paths = []
labels = []

for category in categories:
    category_path = os.path.join(base_path, category)
    for image_name in os.listdir(category_path):
        image_path = os.path.join(category_path, image_name)
        image_paths.append(image_path)
        labels.append(category)
```

```
df = pd.DataFrame({
    "image_path": image_paths,
    "label": labels
})
```

```
df
```

```
                                             image_path         label
0     /kaggle/input/covid19-lung-ct-scans/COVID-19_L...      COVID-19
1     /kaggle/input/covid19-lung-ct-scans/COVID-19_L...      COVID-19
2     /kaggle/input/covid19-lung-ct-scans/COVID-19_L...      COVID-19
3     /kaggle/input/covid19-lung-ct-scans/COVID-19_L...      COVID-19
4     /kaggle/input/covid19-lung-ct-scans/COVID-19_L...      COVID-19
...                                                 ...           ...
8435  /kaggle/input/covid19-lung-ct-scans/COVID-19_L...  Non-COVID-19
8436  /kaggle/input/covid19-lung-ct-scans/COVID-19_L...  Non-COVID-19
8437  /kaggle/input/covid19-lung-ct-scans/COVID-19_L...  Non-COVID-19
8438  /kaggle/input/covid19-lung-ct-scans/COVID-19_L...  Non-COVID-19
8439  /kaggle/input/covid19-lung-ct-scans/COVID-19_L...  Non-COVID-19

[8440 rows x 2 columns]
```

```
df.shape
```

```
(8440, 2)
```

```
df.columns
```

```
Index(['image_path', 'label'], dtype='object')
```

```
df.duplicated().sum()
```

```
0
```

```
df.isnull().sum()
```

```
image_path    0
label         0
dtype: int64
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8440 entries, 0 to 8439
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   image_path  8440 non-null   object
 1   label       8440 non-null   object
dtypes: object(2)
memory usage: 132.0+ KB
```

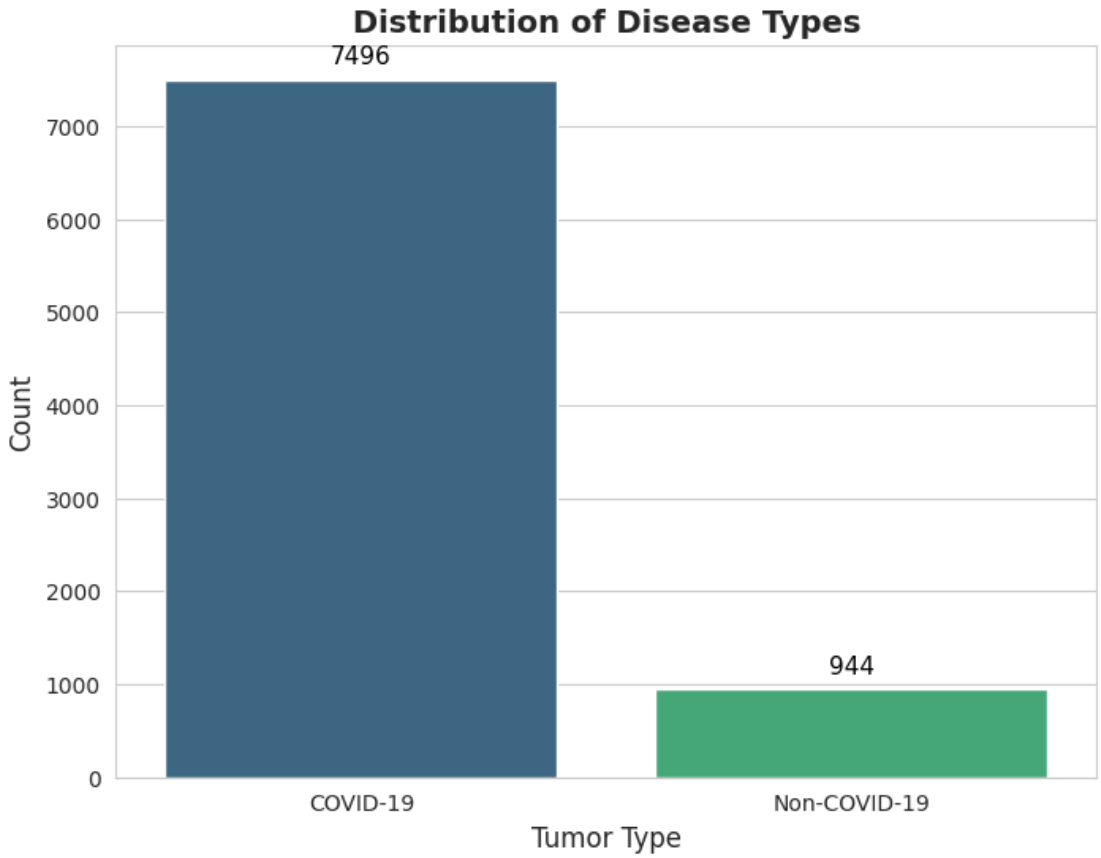
```
df['label'].unique()
```

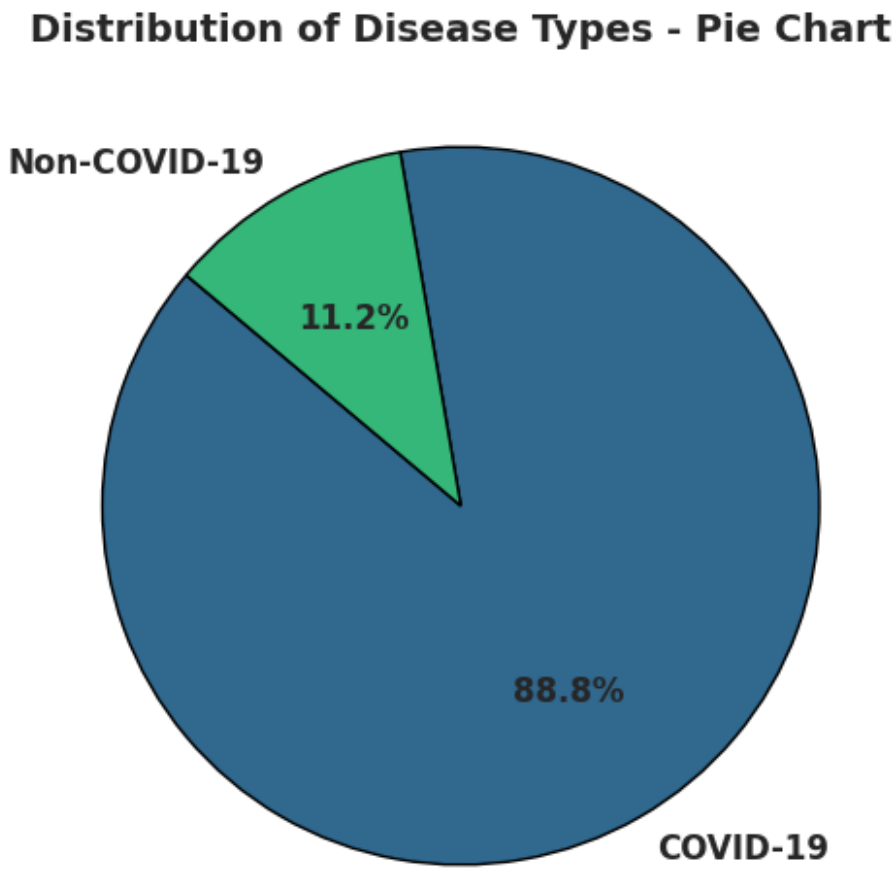
```
array(['COVID-19', 'Non-COVID-19'], dtype=object)
```

```
df['label'].value_counts()
```

```
label
COVID-19        7496
Non-COVID-19     944
Name: count, dtype: int64
```

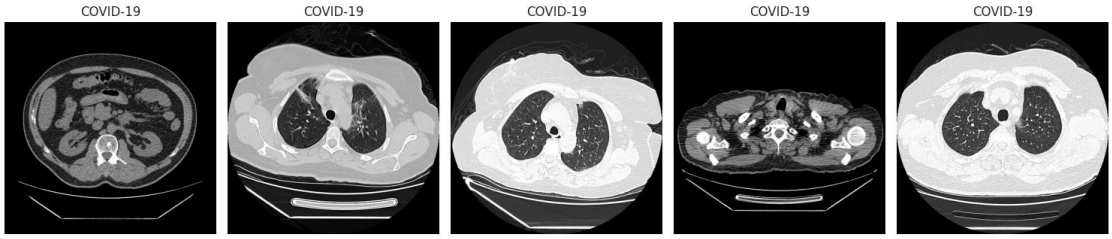
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_style("whitegrid")

fig, ax = plt.subplots(figsize=(8, 6))
sns.countplot(data=df, x="label", palette="viridis", ax=ax)

ax.set_title("Distribution of Disease Types", fontsize=14, fontweight='bold')
ax.set_xlabel("Tumor Type", fontsize=12)
ax.set_ylabel("Count", fontsize=12)

for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='bottom', fontsize=11, color='black',
                xytext=(0, 5), textcoords='offset points')

plt.show()

label_counts = df["label"].value_counts()

fig, ax = plt.subplots(figsize=(8, 6))
colors = sns.color_palette("viridis", len(label_counts))

ax.pie(label_counts, labels=label_counts.index, autopct='%1.1f%%',
       startangle=140, colors=colors, textprops={'fontsize': 12, 'weight':
'bold'},
       wedgeprops={'edgecolor': 'black', 'linewidth': 1})

ax.set_title("Distribution of Disease Types - Pie Chart", fontsize=14,
fontweight='bold')

plt.show()
```

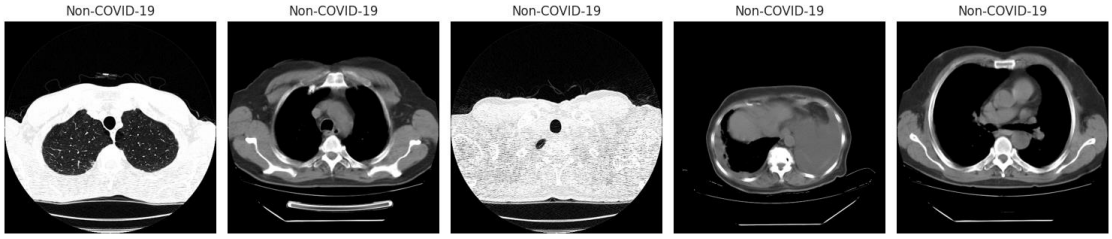

Distribution of Disease Types
7496
944
7000
6000
5000
4000
3000
2000
1000
0
Count
COVID-19
Non-COVID-19
Tumor Type




```
import cv2

num_images = 5

plt.figure(figsize=(15, 12))

for i, category in enumerate(categories):
    category_images = df[df['label'] ==
category]['image_path'].iloc[:num_images]

    for j, img_path in enumerate(category_images):

        img = cv2.imread(img_path)
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        plt.subplot(len(categories), num_images, i * num_images + j + 1)
        plt.imshow(img)
        plt.axis('off')
        plt.title(category)
```

```
plt.tight_layout()
plt.show()
```






```
max_samples = df['label'].value_counts().max()

balanced_df = df.groupby('label', group_keys=False).apply(
    lambda x: x.sample(n=max_samples, replace=True, random_state=42)
).reset_index(drop=True)

balanced_df = balanced_df[['image_path', 'label']]
```

```
/tmp/ipykernel_37/2249685847.py:3: DeprecationWarning: DataFrameGroupBy.apply
operated on the grouping columns. This behavior is deprecated, and in a
future version of pandas the grouping columns will be excluded from the
operation. Either pass `include_groups=False` to exclude the groupings or
explicitly select the grouping columns after groupby to silence this warning.
  balanced_df = df.groupby('label', group_keys=False).apply(
```

```
df = balanced_df

df
```

```
                                              image_path         label
0      /kaggle/input/covid19-lung-ct-scans/COVID-19_L...      COVID-19
1      /kaggle/input/covid19-lung-ct-scans/COVID-19_L...      COVID-19
2      /kaggle/input/covid19-lung-ct-scans/COVID-19_L...      COVID-19
3      /kaggle/input/covid19-lung-ct-scans/COVID-19_L...      COVID-19
4      /kaggle/input/covid19-lung-ct-scans/COVID-19_L...      COVID-19
...                                                  ...           ...
14987  /kaggle/input/covid19-lung-ct-scans/COVID-19_L...  Non-COVID-19
14988  /kaggle/input/covid19-lung-ct-scans/COVID-19_L...  Non-COVID-19
14989  /kaggle/input/covid19-lung-ct-scans/COVID-19_L...  Non-COVID-19
```

```
14990  /kaggle/input/covid19-lung-ct-scans/COVID-19_L...  Non-COVID-19
14991  /kaggle/input/covid19-lung-ct-scans/COVID-19_L...  Non-COVID-19

[14992 rows x 2 columns]
```

```python
import numpy as np
import pandas as pd
from PIL import Image
import os
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import joblib
import matplotlib.pyplot as plt
import seaborn as sns

def extract_emf_features(image_path, sample_rate=50000, time_duration=0.5,
                         carrier_frequency=1000, modulation_index=0.8):
    try:
        img = Image.open(image_path).convert('L')
        img_array = np.array(img)
        pixel_data = img_array.flatten().astype(np.float64)
        message_signal_raw = (pixel_data / 127.5) - 1.0
        total_samples = int(sample_rate * time_duration)
        data_length = len(message_signal_raw)
        if data_length == 0:
            return None
        original_indices = np.linspace(0, total_samples - 1, data_length)
        interp_indices = np.arange(total_samples)
        message_signal = np.interp(interp_indices, original_indices,
message_signal_raw)
        time = np.linspace(0, time_duration, total_samples, endpoint=False)
        carrier_wave = np.sin(2 * np.pi * carrier_frequency * time)
        modulated_wave = (1 + modulation_index * message_signal) *
carrier_wave
        modulated_wave = np.clip(modulated_wave, -1.5, 1.5)
        features = [
            np.mean(modulated_wave),
            np.std(modulated_wave),
            np.max(modulated_wave),
            np.min(modulated_wave),
            np.sqrt(np.mean(modulated_wave**2)),
            np.mean(np.abs(modulated_wave)),
            np.var(modulated_wave),
            np.max(np.abs(np.diff(modulated_wave))),
            np.mean(np.abs(np.diff(modulated_wave))),
            np.corrcoef(modulated_wave, message_signal)[0,1] if
```

```
len(message_signal) > 1 else 0
        ]
        return np.array(features)
    except Exception:
        return None

def process_dataset(df, max_samples_per_class=None):
    features_list = []
    labels_list = []
    covid_paths = df[df['label'] == 'COVID-19']['image_path'].tolist()
    non_covid_paths = df[df['label'] == 'Non-COVID-
19']['image_path'].tolist()
    limit = max_samples_per_class if max_samples_per_class is not None else
float('inf')
    min_samples = min(len(covid_paths), len(non_covid_paths), limit)
    print(f"Processing {min_samples} samples per class...")
    for i, path in enumerate(covid_paths[:min_samples]):
        features = extract_emf_features(path)
        if features is not None:
            features_list.append(features)
            labels_list.append(0)
        if (i + 1) % 500 == 0:
            print(f"Processed {i + 1}/{min_samples} COVID-19 images")
    for i, path in enumerate(non_covid_paths[:min_samples]):
        features = extract_emf_features(path)
        if features is not None:
            features_list.append(features)
            labels_list.append(1)
        if (i + 1) % 500 == 0:
            print(f"Processed {i + 1}/{min_samples} Non-COVID-19 images")
    return np.array(features_list), np.array(labels_list)

print("Dataset shape:", df.shape)
print("Class distribution:")
print(df['label'].value_counts())

X, y = process_dataset(df, max_samples_per_class=8000)

if len(X) == 0:
    raise SystemExit("No valid features extracted. Check image paths.")

print(f"Extracted features shape: {X.shape}")
print(f"Labels shape: {y.shape}")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

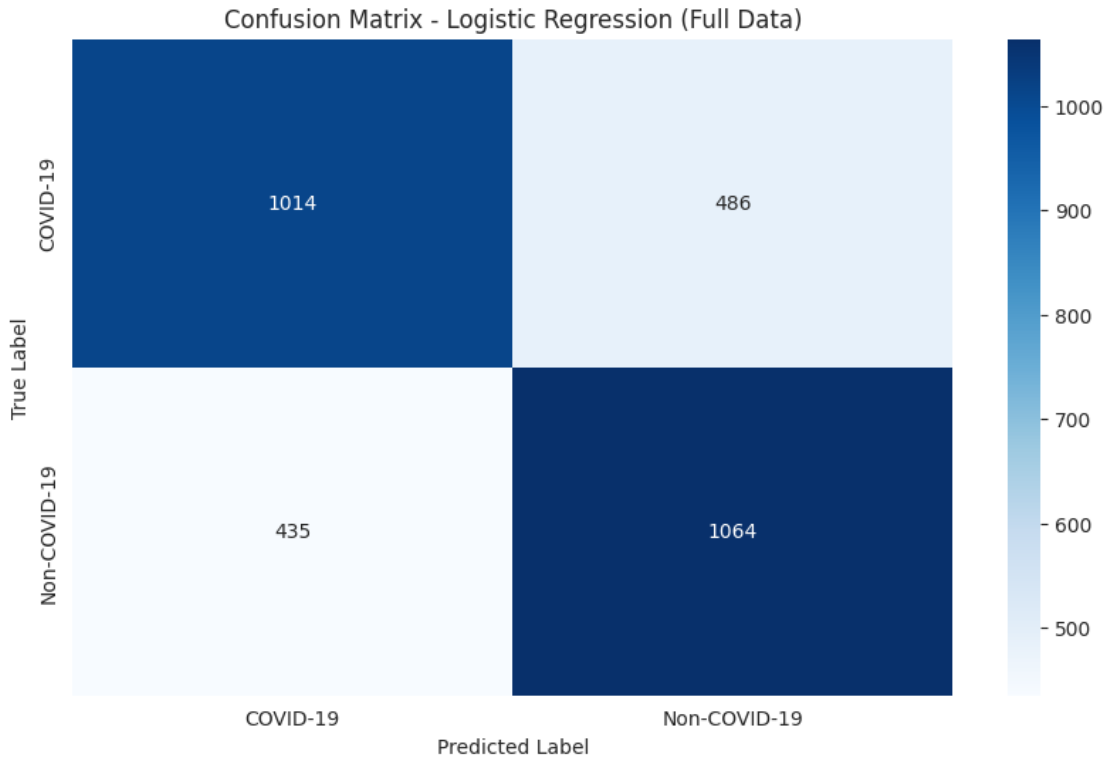
```
X_test_scaled = scaler.transform(X_test)

logistic_model = LogisticRegression(random_state=42, max_iter=1000,
solver='lbfgs')
logistic_model.fit(X_train_scaled, y_train)

y_pred = logistic_model.predict(X_test_scaled)
y_pred_proba = logistic_model.predict_proba(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)

print("\n--- Logistic Regression Results (Full Data) ---")
print(f"Accuracy: {accuracy:.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['COVID-19', 'Non-
COVID-19']))

coefficients = pd.DataFrame({
    'feature': [f'feature_{i}' for i in range(X.shape[1])],
    'coefficient': logistic_model.coef_[0],
    'abs_coefficient': np.abs(logistic_model.coef_[0])
}).sort_values('abs_coefficient', ascending=False)

print("Top 5 Most Important Features (by absolute coefficient):")
print(coefficients.head())
print(f"\nIntercept: {logistic_model.intercept_[0]:.4f}")

cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', LogisticRegression(random_state=42, max_iter=1000))
])

pipeline.fit(X_train, y_train)
joblib.dump(pipeline, 'covid_emf_logistic_classifier_full.pkl')
joblib.dump(scaler, 'feature_scaler_full.pkl')
print("\nPipeline model saved as 'covid_emf_logistic_classifier_full.pkl'")

def predict_single_image(image_path, pipeline):
    features = extract_emf_features(image_path)
    if features is not None:
        features = features.reshape(1, -1)
        prediction = pipeline.predict(features)[0]
        probability = pipeline.predict_proba(features)[0]
        label = 'COVID-19' if prediction == 0 else 'Non-COVID-19'
        return label, probability, features
    return None, None, None
```

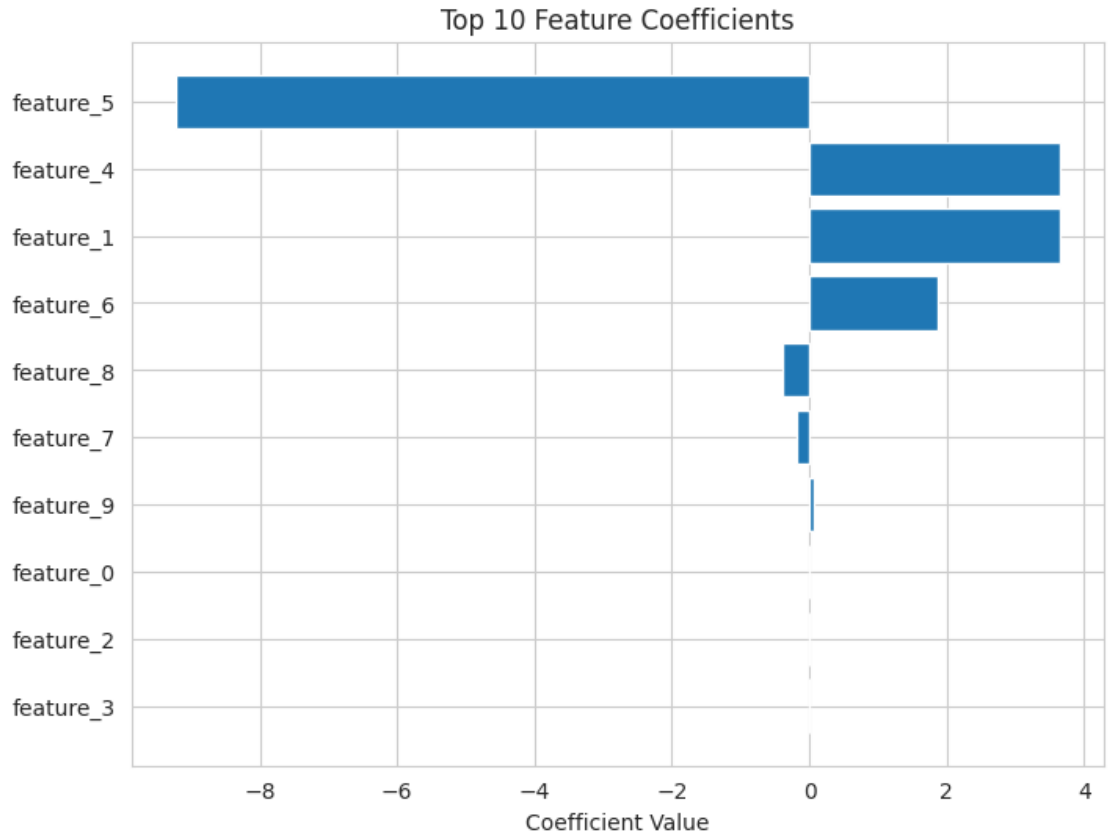
```
sample_path = df['image_path'].iloc[0]
label, prob, feats = predict_single_image(sample_path, pipeline)
if label:
    log_odds = np.log(prob[0]/prob[1])
    print(f"\nSample prediction for {sample_path}:")
    print(f"Predicted: {label}")
    print(f"Probabilities: COVID-19: {prob[0]:.4f}, Non-COVID-19:
{prob[1]:.4f}")
    print(f"Log odds (COVID-19 vs Non-COVID-19): {log_odds:.4f}")

print(f"\nPipeline prediction on test set (Accuracy check):
{pipeline.score(X_test, y_test):.4f}")

plt.figure(figsize=(10, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['COVID-19', 'Non-COVID-19'],
            yticklabels=['COVID-19', 'Non-COVID-19'])
plt.title('Confusion Matrix - Logistic Regression (Full Data)')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

plt.figure(figsize=(8, 6))
top_features = coefficients.head(10)
plt.barh(range(len(top_features)), top_features['coefficient'])
plt.yticks(range(len(top_features)), top_features['feature'])
plt.xlabel('Coefficient Value')
plt.title('Top 10 Feature Coefficients')
plt.gca().invert_yaxis()
plt.show()
```

```
Dataset shape: (14992, 2)
Class distribution:
label
COVID-19        7496
Non-COVID-19    7496
Name: count, dtype: int64
Processing 7496 samples per class...
Processed 500/7496 COVID-19 images
Processed 1000/7496 COVID-19 images
Processed 1500/7496 COVID-19 images
Processed 2000/7496 COVID-19 images
Processed 2500/7496 COVID-19 images
Processed 3000/7496 COVID-19 images
Processed 3500/7496 COVID-19 images
Processed 4000/7496 COVID-19 images
Processed 4500/7496 COVID-19 images
Processed 5000/7496 COVID-19 images
Processed 5500/7496 COVID-19 images
```

```
Processed 6000/7496 COVID-19 images
Processed 6500/7496 COVID-19 images
Processed 7000/7496 COVID-19 images
Processed 500/7496 Non-COVID-19 images
Processed 1000/7496 Non-COVID-19 images
Processed 1500/7496 Non-COVID-19 images
Processed 2000/7496 Non-COVID-19 images
Processed 2500/7496 Non-COVID-19 images
Processed 3000/7496 Non-COVID-19 images
Processed 3500/7496 Non-COVID-19 images
Processed 4000/7496 Non-COVID-19 images
Processed 4500/7496 Non-COVID-19 images
Processed 5000/7496 Non-COVID-19 images
Processed 5500/7496 Non-COVID-19 images
Processed 6000/7496 Non-COVID-19 images
Processed 6500/7496 Non-COVID-19 images
Processed 7000/7496 Non-COVID-19 images
Extracted features shape: (14992, 10)
Labels shape: (14992,)

--- Logistic Regression Results (Full Data) ---
Accuracy: 0.6929

Classification Report:
              precision    recall  f1-score   support

    COVID-19       0.70      0.68      0.69      1500
Non-COVID-19       0.69      0.71      0.70      1499

    accuracy                           0.69      2999
   macro avg       0.69      0.69      0.69      2999
weighted avg       0.69      0.69      0.69      2999

Top 5 Most Important Features (by absolute coefficient):
     feature  coefficient  abs_coefficient
5  feature_5    -9.218275         9.218275
4  feature_4     3.646222         3.646222
1  feature_1     3.644536         3.644536
6  feature_6     1.875163         1.875163
8  feature_8    -0.380225         0.380225

Intercept: 0.0025

Confusion Matrix:
[[1014  486]
 [ 435 1064]]

Pipeline model saved as 'covid_emf_logistic_classifier_full.pkl'
```

```
Sample prediction for /kaggle/input/covid19-lung-ct-scans/COVID-
19_Lung_CT_Scans/COVID-19/COVID-19_0860.png:
Predicted: COVID-19
Probabilities: COVID-19: 0.8185, Non-COVID-19: 0.1815
Log odds (COVID-19 vs Non-COVID-19): 1.5060

Pipeline prediction on test set (Accuracy check): 0.6929
```






```
import numpy as np
import pandas as pd
from PIL import Image
import os
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import joblib
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.tree import plot_tree

def extract_emf_features(image_path, sample_rate=50000, time_duration=0.5,
                         carrier_frequency=1000, modulation_index=0.8):
    try:
        img = Image.open(image_path).convert('L')
        img_array = np.array(img)
        pixel_data = img_array.flatten().astype(np.float64)
        message_signal_raw = (pixel_data / 127.5) - 1.0
        total_samples = int(sample_rate * time_duration)
        data_length = len(message_signal_raw)
```

```
        if data_length == 0:
            return None
        original_indices = np.linspace(0, total_samples - 1, data_length)
        interp_indices = np.arange(total_samples)
        message_signal = np.interp(interp_indices, original_indices,
message_signal_raw)
        time = np.linspace(0, time_duration, total_samples, endpoint=False)
        carrier_wave = np.sin(2 * np.pi * carrier_frequency * time)
        modulated_wave = (1 + modulation_index * message_signal) *
carrier_wave
        modulated_wave = np.clip(modulated_wave, -1.5, 1.5)
        features = [
            np.mean(modulated_wave),
            np.std(modulated_wave),
            np.max(modulated_wave),
            np.min(modulated_wave),
            np.sqrt(np.mean(modulated_wave**2)),
            np.mean(np.abs(modulated_wave)),
            np.var(modulated_wave),
            np.max(np.abs(np.diff(modulated_wave))),
            np.mean(np.abs(np.diff(modulated_wave))),
            np.corrcoef(modulated_wave, message_signal)[0,1] if
len(message_signal) > 1 else 0
        ]
        return np.array(features)
    except Exception:
        return None

def process_dataset(df, max_samples_per_class=None):
    features_list = []
    labels_list = []
    covid_paths = df[df['label'] == 'COVID-19']['image_path'].tolist()
    non_covid_paths = df[df['label'] == 'Non-COVID-
19']['image_path'].tolist()
    limit = max_samples_per_class if max_samples_per_class is not None else
float('inf')
    min_samples = min(len(covid_paths), len(non_covid_paths), limit)
    print(f"Processing {min_samples} samples per class...")
    for i, path in enumerate(covid_paths[:min_samples]):
        features = extract_emf_features(path)
        if features is not None:
            features_list.append(features)
            labels_list.append(0)
        if (i + 1) % 500 == 0:
            print(f"Processed {i + 1}/{min_samples} COVID-19 images")
    for i, path in enumerate(non_covid_paths[:min_samples]):
        features = extract_emf_features(path)
        if features is not None:
            features_list.append(features)
            labels_list.append(1)
```

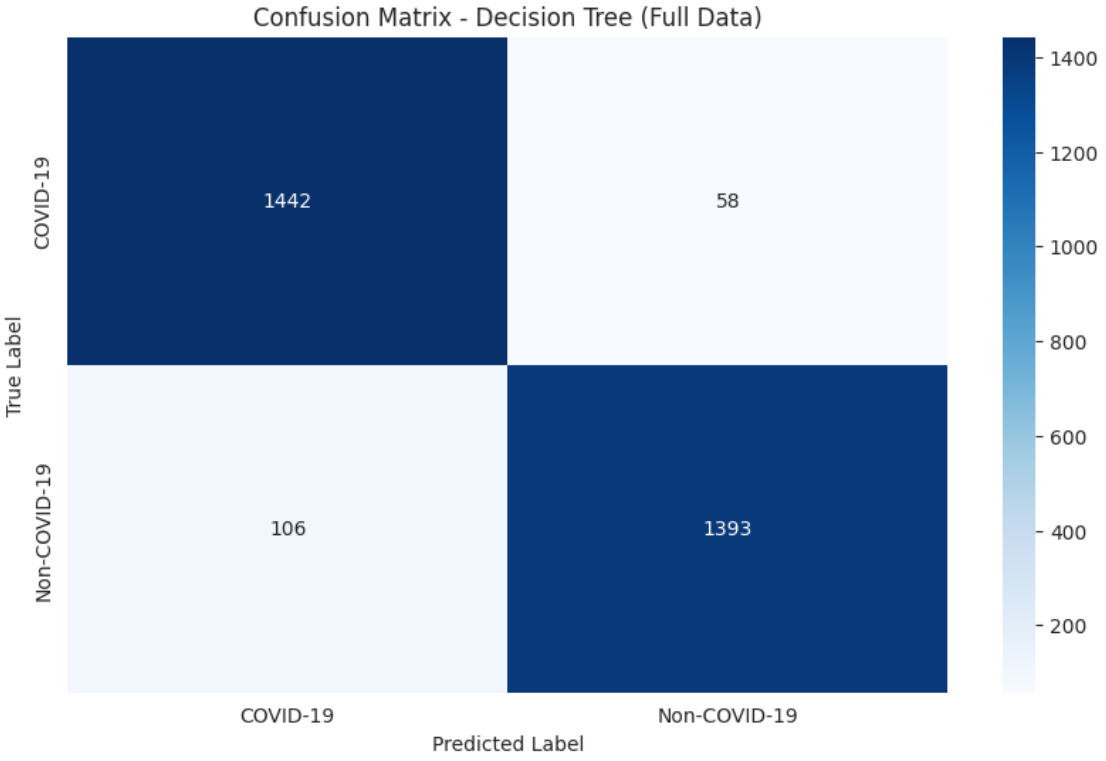
```
        if (i + 1) % 500 == 0:
            print(f"Processed {i + 1}/{min_samples} Non-COVID-19 images")
    return np.array(features_list), np.array(labels_list)

try:
    print("Dataset shape:", df.shape)
    print("Class distribution:")
    print(df['label'].value_counts())
except NameError:
    raise SystemExit("DataFrame 'df' is not defined. Please provide a valid
DataFrame with 'image_path' and 'label' columns.")

X, y = process_dataset(df, max_samples_per_class=8000)

if len(X) == 0:
    raise SystemExit("No valid features extracted. Check image paths.")

print(f"Extracted features shape: {X.shape}")
print(f"Labels shape: {y.shape}")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

decision_tree = DecisionTreeClassifier(random_state=42, max_depth=10)
decision_tree.fit(X_train_scaled, y_train)

y_pred = decision_tree.predict(X_test_scaled)
y_pred_proba = decision_tree.predict_proba(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)

print("\n--- Decision Tree Results (Full Data) ---")
print(f"Accuracy: {accuracy:.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['COVID-19', 'Non-
COVID-19']))

feature_names = [f'feature_{i}' for i in range(X.shape[1])]
importance = pd.DataFrame({
    'feature': feature_names,
    'importance': decision_tree.feature_importances_
}).sort_values('importance', ascending=False)

print("\nTop 5 Most Important Features (by importance score):")
print(importance.head())
```

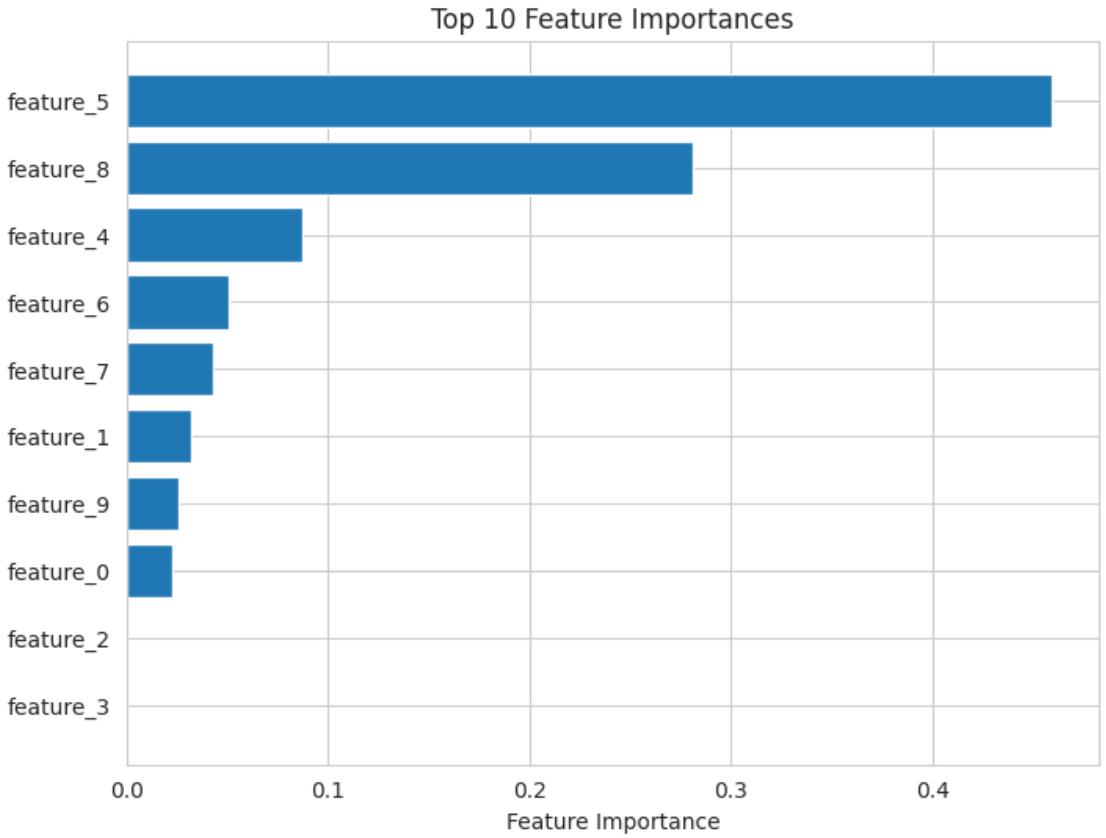
```
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', DecisionTreeClassifier(random_state=42, max_depth=10))
])

pipeline.fit(X_train, y_train)
joblib.dump(pipeline, 'covid_emf_decision_tree_classifier_full.pkl')
joblib.dump(scaler, 'feature_scaler_full.pkl')
print("\nPipeline model saved as
'covid_emf_decision_tree_classifier_full.pkl'")

def predict_single_image(image_path, pipeline):
    features = extract_emf_features(image_path)
    if features is not None:
        features = features.reshape(1, -1)
        prediction = pipeline.predict(features)[0]
        probability = pipeline.predict_proba(features)[0]
        label = 'COVID-19' if prediction == 0 else 'Non-COVID-19'
        return label, probability, features
    return None, None, None

sample_path = df['image_path'].iloc[0]
label, prob, feats = predict_single_image(sample_path, pipeline)
if label:
    log_odds = np.log(prob[0]/prob[1]) if prob[1] != 0 else float('inf')
    print(f"\nSample prediction for {sample_path}:")
    print(f"Predicted: {label}")
    print(f"Probabilities: COVID-19: {prob[0]:.4f}, Non-COVID-19:
{prob[1]:.4f}")
    print(f"Log odds (COVID-19 vs Non-COVID-19): {log_odds:.4f}")

print(f"\nPipeline prediction on test set (Accuracy check):
{pipeline.score(X_test, y_test):.4f}")

plt.figure(figsize=(10, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['COVID-19', 'Non-COVID-19'],
            yticklabels=['COVID-19', 'Non-COVID-19'])
plt.title('Confusion Matrix - Decision Tree (Full Data)')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

plt.figure(figsize=(8, 6))
top_features = importance.head(10)
```

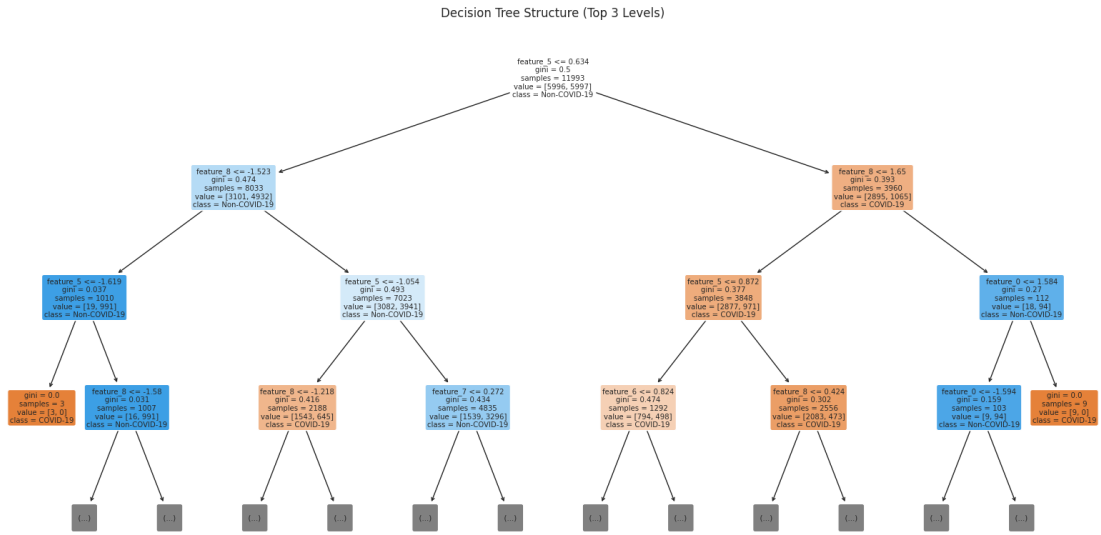
```
plt.barh(range(len(top_features)), top_features['importance'])
plt.yticks(range(len(top_features)), top_features['feature'])
plt.xlabel('Feature Importance')
plt.title('Top 10 Feature Importances')
plt.gca().invert_yaxis()
plt.show()

plt.figure(figsize=(20, 10))
plot_tree(decision_tree, feature_names=feature_names, class_names=['COVID-
19', 'Non-COVID-19'],
          filled=True, rounded=True, max_depth=3)
plt.title('Decision Tree Structure (Top 3 Levels)')
plt.show()
```

```
Dataset shape: (14992, 2)
Class distribution:
label
COVID-19        7496
Non-COVID-19    7496
Name: count, dtype: int64
Processing 7496 samples per class...
Processed 500/7496 COVID-19 images
Processed 1000/7496 COVID-19 images
Processed 1500/7496 COVID-19 images
Processed 2000/7496 COVID-19 images
Processed 2500/7496 COVID-19 images
Processed 3000/7496 COVID-19 images
Processed 3500/7496 COVID-19 images
Processed 4000/7496 COVID-19 images
Processed 4500/7496 COVID-19 images
Processed 5000/7496 COVID-19 images
Processed 5500/7496 COVID-19 images
Processed 6000/7496 COVID-19 images
Processed 6500/7496 COVID-19 images
Processed 7000/7496 COVID-19 images
Processed 500/7496 Non-COVID-19 images
Processed 1000/7496 Non-COVID-19 images
Processed 1500/7496 Non-COVID-19 images
Processed 2000/7496 Non-COVID-19 images
Processed 2500/7496 Non-COVID-19 images
Processed 3000/7496 Non-COVID-19 images
Processed 3500/7496 Non-COVID-19 images
Processed 4000/7496 Non-COVID-19 images
Processed 4500/7496 Non-COVID-19 images
Processed 5000/7496 Non-COVID-19 images
Processed 5500/7496 Non-COVID-19 images
Processed 6000/7496 Non-COVID-19 images
Processed 6500/7496 Non-COVID-19 images
Processed 7000/7496 Non-COVID-19 images
Extracted features shape: (14992, 10)
```

```
Labels shape: (14992,)

--- Decision Tree Results (Full Data) ---
Accuracy: 0.9453

Classification Report:
              precision    recall  f1-score   support

    COVID-19       0.93      0.96      0.95      1500
Non-COVID-19       0.96      0.93      0.94      1499

    accuracy                           0.95      2999
   macro avg       0.95      0.95      0.95      2999
weighted avg       0.95      0.95      0.95      2999


Top 5 Most Important Features (by importance score):
     feature  importance
5  feature_5    0.459745
8  feature_8    0.280992
4  feature_4    0.086989
6  feature_6    0.050395
7  feature_7    0.042537

Confusion Matrix:
[[1442   58]
 [ 106 1393]]

Pipeline model saved as 'covid_emf_decision_tree_classifier_full.pkl'

Sample prediction for /kaggle/input/covid19-lung-ct-scans/COVID-
19_Lung_CT_Scans/COVID-19/COVID-19_0860.png:
Predicted: COVID-19
Probabilities: COVID-19: 0.9291, Non-COVID-19: 0.0709
Log odds (COVID-19 vs Non-COVID-19): 2.5728

Pipeline prediction on test set (Accuracy check): 0.9453
```


Confusion Matrix - Decision Tree (Full Data)
1442
58
106
1393
COVID-19
Non-COVID-19
True Label
COVID-19
Non-COVID-19
Predicted Label
1400
1200
1000
800
600
400
200


Top 10 Feature Importances
feature_5
feature_8
feature_4
feature_6
feature_7
feature_1
feature_9
feature_0
feature_2
feature_3
0.0
0.1
0.2
0.3
0.4
Feature Importance




```
import numpy as np
import pandas as pd
from PIL import Image
import os
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import joblib
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.tree import plot_tree
import cv2

def apply_clahe(img_array):
    img_8bit = img_array.astype(np.uint8)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
    clahe_img = clahe.apply(img_8bit)
    return clahe_img.astype(np.float64)

def extract_emf_features_from_signal(pixel_data, sample_rate=50000,
time_duration=0.5,
                                     carrier_frequency=1000,
modulation_index=0.8):
    message_signal_raw = (pixel_data / 127.5) - 1.0
    total_samples = int(sample_rate * time_duration)
    data_length = len(message_signal_raw)
    if data_length == 0:
        return np.zeros(10)
    original_indices = np.linspace(0, total_samples - 1, data_length)
```

```
    interp_indices = np.arange(total_samples)
    message_signal = np.interp(interp_indices, original_indices,
message_signal_raw)
    time = np.linspace(0, time_duration, total_samples, endpoint=False)
    carrier_wave = np.sin(2 * np.pi * carrier_frequency * time)
    modulated_wave = (1 + modulation_index * message_signal) * carrier_wave
    modulated_wave = np.clip(modulated_wave, -1.5, 1.5)
    features = [
        np.mean(modulated_wave), np.std(modulated_wave),
np.max(modulated_wave),
        np.min(modulated_wave), np.sqrt(np.mean(modulated_wave**2)),
        np.mean(np.abs(modulated_wave)), np.var(modulated_wave),
        np.max(np.abs(np.diff(modulated_wave))),
np.mean(np.abs(np.diff(modulated_wave))),
        np.corrcoef(modulated_wave, message_signal)[0,1] if
len(message_signal) > 1 else 0
    ]
    return np.array(features)

def extract_h_w_c_emf_features(image_path):
    try:
        img = Image.open(image_path).convert('L')
        img_array = np.array(img)
        clahe_img_array = apply_clahe(img_array)
        signal_H = clahe_img_array.flatten()
        features_H = extract_emf_features_from_signal(signal_H)
        signal_W = clahe_img_array.T.flatten()
        features_W = extract_emf_features_from_signal(signal_W)
        features_C = (features_H + features_W) / 2
        final_features = np.concatenate([features_H, features_W, features_C])
        if final_features.shape[0] != 30:
            raise ValueError("Feature vector is not 30 elements long.")
        return final_features
    except Exception as e:
        print(f"Error processing {image_path}: {e}")
        return None

def process_dataset(df, max_samples_per_class=None):
    features_list = []
    labels_list = []
    covid_paths = df[df['label'] == 'COVID-19']['image_path'].tolist()
    non_covid_paths = df[df['label'] == 'Non-COVID-
19']['image_path'].tolist()
    limit = max_samples_per_class if max_samples_per_class is not None else
float('inf')
    min_samples = min(len(covid_paths), len(non_covid_paths), limit)
    print(f"Processing {min_samples} samples per class with CLAHE/HWC...")
    for i, path in enumerate(covid_paths[:min_samples]):
        features = extract_h_w_c_emf_features(path)
        if features is not None and features.shape[0] == 30:
```

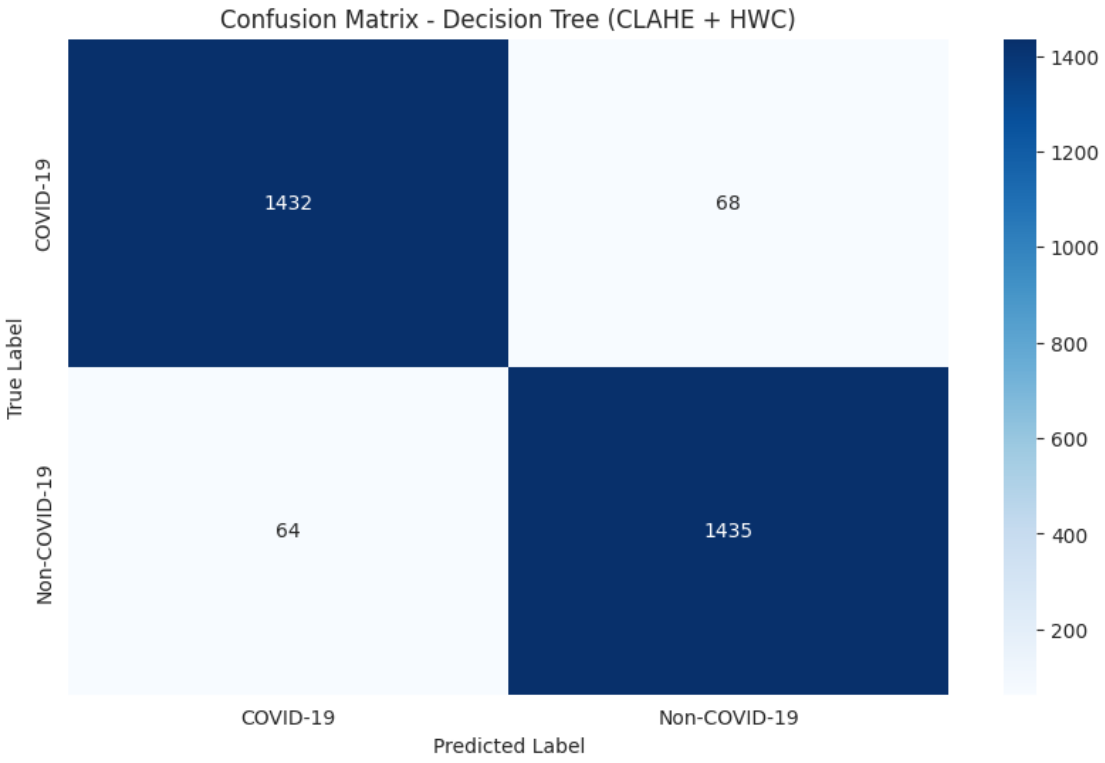
```
            features_list.append(features)
            labels_list.append(0)
        if (i + 1) % 500 == 0:
            print(f"Processed {i + 1}/{min_samples} COVID-19 images")
    for i, path in enumerate(non_covid_paths[:min_samples]):
        features = extract_h_w_c_emf_features(path)
        if features is not None and features.shape[0] == 30:
            features_list.append(features)
            labels_list.append(1)
        if (i + 1) % 500 == 0:
            print(f"Processed {i + 1}/{min_samples} Non-COVID-19 images")
    return np.array(features_list), np.array(labels_list)

try:
    print("Dataset shape:", df.shape)
    print("Class distribution:")
    print(df['label'].value_counts())
except NameError:
    raise SystemExit("DataFrame 'df' is not defined. Please ensure it's
available.")

X, y = process_dataset(df, max_samples_per_class=8000)

if len(X) == 0:
    raise SystemExit("No valid features extracted. Check image paths or CLAHE
setup.")

print(f"Extracted features shape: {X.shape} (Now 30 features: 10 H + 10 W +
10 C)")
print(f"Labels shape: {y.shape}")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

decision_tree = DecisionTreeClassifier(random_state=42, max_depth=10)
decision_tree.fit(X_train_scaled, y_train)

y_pred = decision_tree.predict(X_test_scaled)
y_pred_proba = decision_tree.predict_proba(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)

print("\n--- Decision Tree Results (CLAHE + HWC EMF Features) ---")
print(f"Accuracy: {accuracy:.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['COVID-19', 'Non-
```

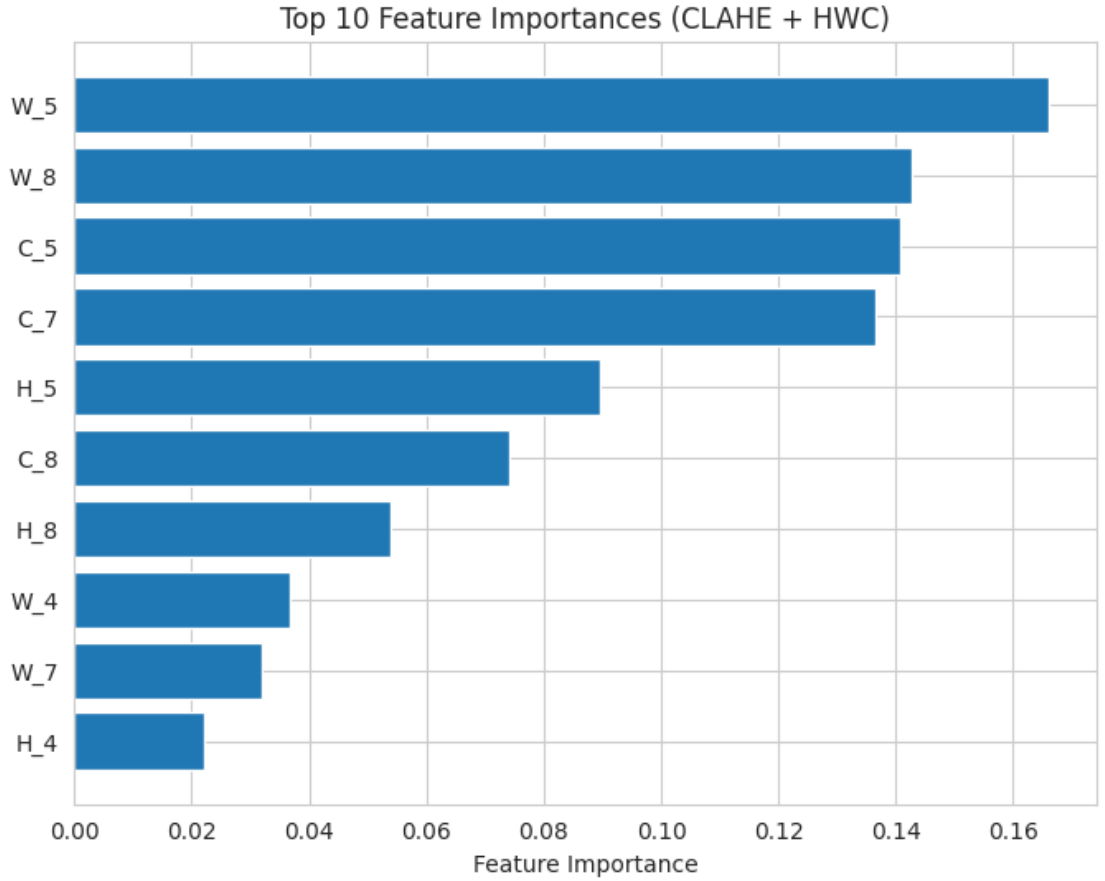
```
COVID-19']))

feature_names = [f'H_{i}' for i in range(10)] + [f'W_{i}' for i in range(10)]
+ [f'C_{i}' for i in range(10)]
importance = pd.DataFrame({
    'feature': feature_names,
    'importance': decision_tree.feature_importances_
}).sort_values('importance', ascending=False)

print("\nTop 5 Most Important Features (by importance score):")
print(importance.head())

cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', DecisionTreeClassifier(random_state=42, max_depth=10))
])

pipeline.fit(X_train, y_train)
joblib.dump(pipeline, 'covid_emf_decision_tree_clahe_hwc.pkl')
joblib.dump(scaler, 'feature_scaler_clahe_hwc.pkl')
print("\nPipeline model saved as 'covid_emf_decision_tree_clahe_hwc.pkl'")

def predict_single_image(image_path, pipeline):
    features = extract_h_w_c_emf_features(image_path)
    if features is not None and features.shape[0] == 30:
        features = features.reshape(1, -1)
        prediction = pipeline.predict(features)[0]
        probability = pipeline.predict_proba(features)[0]
        label = 'COVID-19' if prediction == 0 else 'Non-COVID-19'
        return label, probability, features
    return None, None, None

sample_path = df['image_path'].iloc[0]
label, prob, feats = predict_single_image(sample_path, pipeline)
if label:
    log_odds = np.log(prob[0]/prob[1]) if prob[1] != 0 else float('inf')
    print(f"\nSample prediction for {sample_path}:")
    print(f"Predicted: {label}")
    print(f"Probabilities: COVID-19: {prob[0]:.4f}, Non-COVID-19:
{prob[1]:.4f}")
    print(f"Log odds (COVID-19 vs Non-COVID-19): {log_odds:.4f}")

print(f"\nPipeline prediction on test set (Accuracy check):
{pipeline.score(X_test, y_test):.4f}")
```

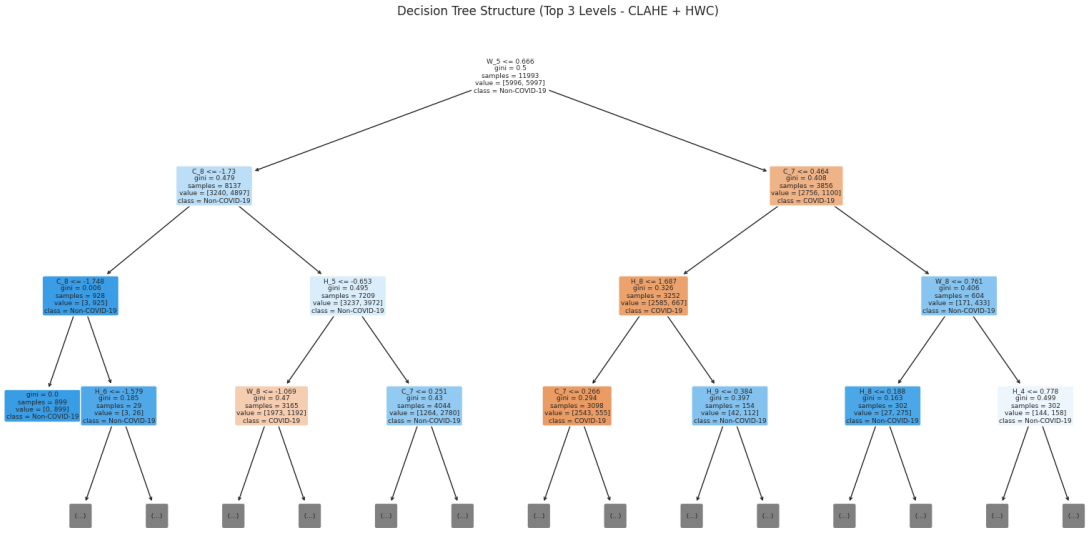
```
plt.figure(figsize=(10, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['COVID-19', 'Non-COVID-19'],
            yticklabels=['COVID-19', 'Non-COVID-19'])
plt.title('Confusion Matrix - Decision Tree (CLAHE + HWC)')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

plt.figure(figsize=(8, 6))
top_features = importance.head(10)
plt.barh(range(len(top_features)), top_features['importance'])
plt.yticks(range(len(top_features)), top_features['feature'])
plt.xlabel('Feature Importance')
plt.title('Top 10 Feature Importances (CLAHE + HWC)')
plt.gca().invert_yaxis()
plt.show()

plt.figure(figsize=(20, 10))
plot_tree(decision_tree, feature_names=feature_names, class_names=['COVID-
19', 'Non-COVID-19'],
          filled=True, rounded=True, max_depth=3)
plt.title('Decision Tree Structure (Top 3 Levels - CLAHE + HWC)')
plt.show()
```

```
Dataset shape: (14992, 2)
Class distribution:
label
COVID-19        7496
Non-COVID-19    7496
Name: count, dtype: int64
Processing 7496 samples per class with CLAHE/HWC...
Processed 500/7496 COVID-19 images
Processed 1000/7496 COVID-19 images
Processed 1500/7496 COVID-19 images
Processed 2000/7496 COVID-19 images
Processed 2500/7496 COVID-19 images
Processed 3000/7496 COVID-19 images
Processed 3500/7496 COVID-19 images
Processed 4000/7496 COVID-19 images
Processed 4500/7496 COVID-19 images
Processed 5000/7496 COVID-19 images
Processed 5500/7496 COVID-19 images
Processed 6000/7496 COVID-19 images
Processed 6500/7496 COVID-19 images
Processed 7000/7496 COVID-19 images
Processed 500/7496 Non-COVID-19 images
Processed 1000/7496 Non-COVID-19 images
Processed 1500/7496 Non-COVID-19 images
Processed 2000/7496 Non-COVID-19 images
```

```
Processed 2500/7496 Non-COVID-19 images
Processed 3000/7496 Non-COVID-19 images
Processed 3500/7496 Non-COVID-19 images
Processed 4000/7496 Non-COVID-19 images
Processed 4500/7496 Non-COVID-19 images
Processed 5000/7496 Non-COVID-19 images
Processed 5500/7496 Non-COVID-19 images
Processed 6000/7496 Non-COVID-19 images
Processed 6500/7496 Non-COVID-19 images
Processed 7000/7496 Non-COVID-19 images
Extracted features shape: (14992, 30) (Now 30 features: 10 H + 10 W + 10 C)
Labels shape: (14992,)

--- Decision Tree Results (CLAHE + HWC EMF Features) ---
Accuracy: 0.9560

Classification Report:
              precision    recall  f1-score   support

    COVID-19       0.96      0.95      0.96      1500
Non-COVID-19       0.95      0.96      0.96      1499

    accuracy                           0.96      2999
   macro avg       0.96      0.96      0.96      2999
weighted avg       0.96      0.96      0.96      2999


Top 5 Most Important Features (by importance score):
   feature  importance
15     W_5    0.166019
18     W_8    0.142655
25     C_5    0.140703
27     C_7    0.136417
5      H_5    0.089501

Confusion Matrix:
[[1432   68]
 [  64 1435]]

Pipeline model saved as 'covid_emf_decision_tree_clahe_hwc.pkl'

Sample prediction for /kaggle/input/covid19-lung-ct-scans/COVID-
19_Lung_CT_Scans/COVID-19/COVID-19_0860.png:
Predicted: COVID-19
Probabilities: COVID-19: 1.0000, Non-COVID-19: 0.0000
Log odds (COVID-19 vs Non-COVID-19): inf

Pipeline prediction on test set (Accuracy check): 0.9560
```


Confusion Matrix - Decision Tree (CLAHE + HWC)
1432
68
64
1435
COVID-19
Non-COVID-19
True Label
COVID-19
Non-COVID-19
Predicted Label
1400
1200
1000
800
600
400
200


Top 10 Feature Importances (CLAHE + HWC)
W_5
W_8
C_5
C_7
H_5
C_8
H_8
W_4
W_7
H_4
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0.14
0.16
Feature Importance




```
import numpy as np
import pandas as pd
from PIL import Image
import os
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import joblib
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.tree import plot_tree
import cv2
from skimage.feature import graycoprops, graycomatrix

def extract_glcm_features(img_array):
    img_8bit = img_array.astype(np.uint8)
    distances = [1]
    angles = [0, np.pi/4, np.pi/2, 3*np.pi/4]
    glcm = graycomatrix(img_8bit, distances=distances, angles=angles,
levels=256, symmetric=True, normed=True)
    properties = ['contrast', 'dissimilarity', 'homogeneity', 'energy',
'correlation', 'ASM', 'mean']
    glcm_features = []
    for prop in properties:
        if prop == 'mean':
            feature_set = np.array([np.mean(img_8bit)] * len(angles))
        else:
            feature_set = graycoprops(glcm, prop).flatten()
        glcm_features.extend(feature_set)
```

```
    return np.array(glcm_features)

def apply_clahe(img_array):
    img_8bit = img_array.astype(np.uint8)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
    clahe_img = clahe.apply(img_8bit)
    return clahe_img.astype(np.float64)

def extract_emf_features_from_signal(pixel_data, sample_rate=50000,
time_duration=0.5, carrier_frequency=1000, modulation_index=0.8):
    message_signal_raw = (pixel_data / 127.5) - 1.0
    total_samples = int(sample_rate * time_duration)
    data_length = len(message_signal_raw)
    if data_length == 0:
        return np.zeros(10)
    original_indices = np.linspace(0, total_samples - 1, data_length)
    interp_indices = np.arange(total_samples)
    message_signal = np.interp(interp_indices, original_indices,
message_signal_raw)
    time = np.linspace(0, time_duration, total_samples, endpoint=False)
    carrier_wave = np.sin(2 * np.pi * carrier_frequency * time)
    modulated_wave = (1 + modulation_index * message_signal) * carrier_wave
    modulated_wave = np.clip(modulated_wave, -1.5, 1.5)
    features = [
        np.mean(modulated_wave), np.std(modulated_wave),
np.max(modulated_wave),
        np.min(modulated_wave), np.sqrt(np.mean(modulated_wave**2)),
        np.mean(np.abs(modulated_wave)), np.var(modulated_wave),
        np.max(np.abs(np.diff(modulated_wave))),
np.mean(np.abs(np.diff(modulated_wave))),
        np.corrcoef(modulated_wave, message_signal)[0,1] if
len(message_signal) > 1 else 0
    ]
    return np.array(features)

def extract_composite_features(image_path):
    try:
        img = Image.open(image_path).convert('L')
        img_array = np.array(img)
        clahe_img_array = apply_clahe(img_array)
        features_H =
extract_emf_features_from_signal(clahe_img_array.flatten())
        features_W =
extract_emf_features_from_signal(clahe_img_array.T.flatten())
        features_C = (features_H + features_W) / 2
        emf_features = np.concatenate([features_H, features_W, features_C])
        glcm_features = extract_glcm_features(clahe_img_array)
        final_features = np.concatenate([emf_features, glcm_features])
        if final_features.shape[0] != 58:
            raise ValueError(f"Feature vector size mismatch: Expected 58, got
```

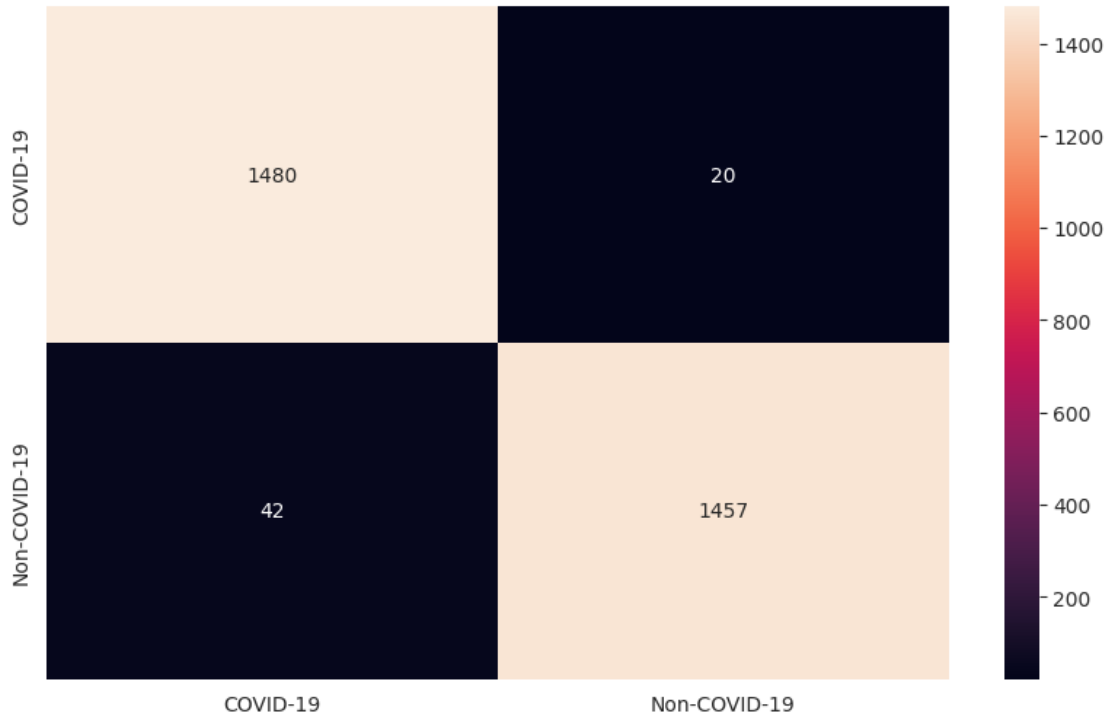
```
{final_features.shape[0]}")
        return final_features
    except Exception as e:
        print(f"Error processing {image_path}: {e}")
        return None

def process_dataset(df, max_samples_per_class=None):
    features_list = []
    labels_list = []
    covid_paths = df[df['label'] == 'COVID-19']['image_path'].tolist()
    non_covid_paths = df[df['label'] == 'Non-COVID-
19']['image_path'].tolist()
    limit = max_samples_per_class if max_samples_per_class is not None else
float('inf')
    min_samples = min(len(covid_paths), len(non_covid_paths), limit)
    print(f"Processing {min_samples} samples per class with CLAHE/HWC-EMF +
GLCM...")
    for i, path in enumerate(covid_paths[:min_samples]):
        features = extract_composite_features(path)
        if features is not None and features.shape[0] == 58:
            features_list.append(features)
            labels_list.append(0)
        if (i + 1) % 500 == 0:
            print(f"Processed {i + 1}/{min_samples} COVID-19 images")
    for i, path in enumerate(non_covid_paths[:min_samples]):
        features = extract_composite_features(path)
        if features is not None and features.shape[0] == 58:
            features_list.append(features)
            labels_list.append(1)
        if (i + 1) % 500 == 0:
            print(f"Processed {i + 1}/{min_samples} Non-COVID-19 images")
    return np.array(features_list), np.array(labels_list)

print("Dataset shape:", df.shape)
print("Class distribution:")
print(df['label'].value_counts())

X, y = process_dataset(df, max_samples_per_class=8000)
if len(X) == 0:
    raise SystemExit("No valid features extracted. Check libraries and
paths.")

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
decision_tree = DecisionTreeClassifier(random_state=42, max_depth=10)
decision_tree.fit(X_train_scaled, y_train)
```

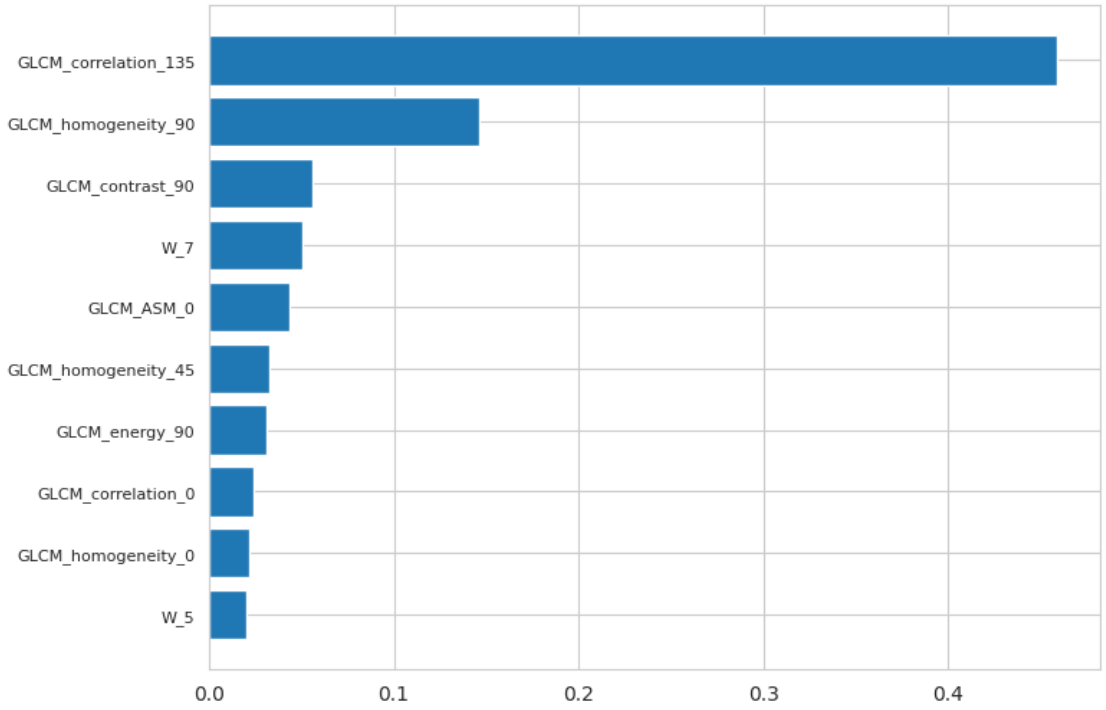
```
y_pred = decision_tree.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f}")
print(classification_report(y_test, y_pred, target_names=['COVID-19', 'Non-
COVID-19']))

emf_names = [f'H_{i}' for i in range(10)] + [f'W_{i}' for i in range(10)] +
[f'C_{i}' for i in range(10)]
glcm_properties = ['contrast', 'dissimilarity', 'homogeneity', 'energy',
'correlation', 'ASM', 'mean']
glcm_angles = ['0', '45', '90', '135']
feature_names = emf_names + [f'GLCM_{p}_{a}' for p in glcm_properties for a
in glcm_angles]

importance = pd.DataFrame({'feature': feature_names, 'importance':
decision_tree.feature_importances_}).sort_values('importance',
ascending=False)
print(importance.head())

cm = confusion_matrix(y_test, y_pred)
print(cm)

pipeline = Pipeline([('scaler', StandardScaler()), ('classifier',
DecisionTreeClassifier(random_state=42, max_depth=10))])
pipeline.fit(X_train, y_train)
joblib.dump(pipeline, 'covid_emf_glcm_decision_tree_58feats.pkl')
joblib.dump(scaler, 'feature_scaler_58feats.pkl')

def predict_single_image(image_path, pipeline):
    features = extract_composite_features(image_path)
    if features is not None and features.shape[0] == 58:
        features = features.reshape(1, -1)
        prediction = pipeline.predict(features)[0]
        probability = pipeline.predict_proba(features)[0]
        label = 'COVID-19' if prediction == 0 else 'Non-COVID-19'
        return label, probability, features
    return None, None, None

sample_path = df['image_path'].iloc[0]
label, prob, _ = predict_single_image(sample_path, pipeline)
if label:
    log_odds = np.log(prob[0]/prob[1]) if prob[1] > 1e-6 else float('inf')
    print(label, prob, log_odds)

plt.figure(figsize=(10, 6))
sns.heatmap(cm, annot=True, fmt='d', xticklabels=['COVID-19', 'Non-COVID-
19'], yticklabels=['COVID-19', 'Non-COVID-19'])
plt.show()
```

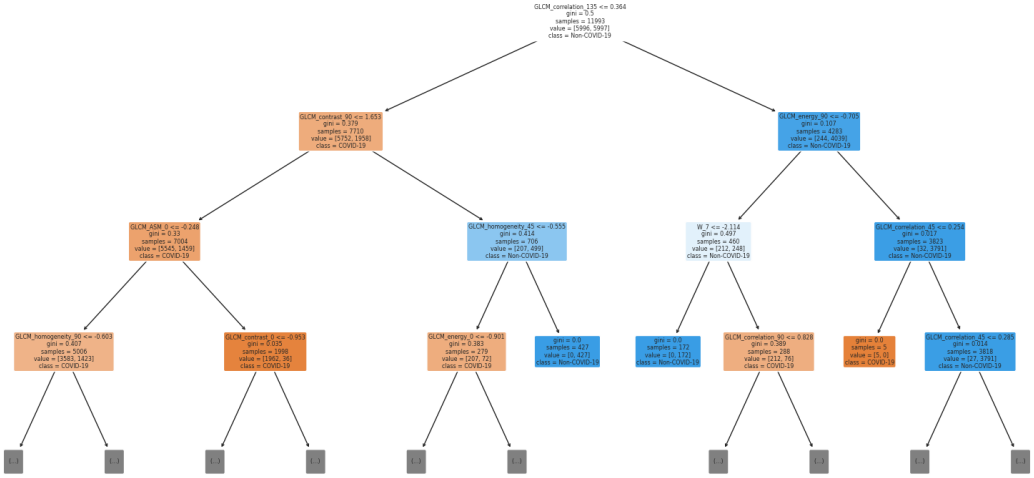
```
plt.figure(figsize=(8, 6))
top_features = importance.head(10)
plt.barh(range(len(top_features)), top_features['importance'])
plt.yticks(range(len(top_features)), top_features['feature'], fontsize=8)
plt.gca().invert_yaxis()
plt.show()

plt.figure(figsize=(20, 10))
plot_tree(decision_tree, feature_names=feature_names, class_names=['COVID-
19', 'Non-COVID-19'], filled=True, rounded=True, max_depth=3)
plt.show()
```

```
Dataset shape: (14992, 2)
Class distribution:
label
COVID-19        7496
Non-COVID-19    7496
Name: count, dtype: int64
Processing 7496 samples per class with CLAHE/HWC-EMF + GLCM...
Processed 500/7496 COVID-19 images
Processed 1000/7496 COVID-19 images
Processed 1500/7496 COVID-19 images
Processed 2000/7496 COVID-19 images
Processed 2500/7496 COVID-19 images
Processed 3000/7496 COVID-19 images
Processed 3500/7496 COVID-19 images
Processed 4000/7496 COVID-19 images
Processed 4500/7496 COVID-19 images
Processed 5000/7496 COVID-19 images
Processed 5500/7496 COVID-19 images
Processed 6000/7496 COVID-19 images
Processed 6500/7496 COVID-19 images
Processed 7000/7496 COVID-19 images
Processed 500/7496 Non-COVID-19 images
Processed 1000/7496 Non-COVID-19 images
Processed 1500/7496 Non-COVID-19 images
Processed 2000/7496 Non-COVID-19 images
Processed 2500/7496 Non-COVID-19 images
Processed 3000/7496 Non-COVID-19 images
Processed 3500/7496 Non-COVID-19 images
Processed 4000/7496 Non-COVID-19 images
Processed 4500/7496 Non-COVID-19 images
Processed 5000/7496 Non-COVID-19 images
Processed 5500/7496 Non-COVID-19 images
Processed 6000/7496 Non-COVID-19 images
Processed 6500/7496 Non-COVID-19 images
Processed 7000/7496 Non-COVID-19 images
Accuracy: 0.9793
              precision    recall  f1-score   support
```

```
    COVID-19       0.97      0.99      0.98      1500
Non-COVID-19       0.99      0.97      0.98      1499

    accuracy                           0.98      2999
   macro avg       0.98      0.98      0.98      2999
weighted avg       0.98      0.98      0.98      2999

                 feature  importance
49  GLCM_correlation_135    0.459105
40   GLCM_homogeneity_90    0.146063
32      GLCM_contrast_90    0.055953
17                   W_7    0.050532
50            GLCM_ASM_0    0.043301
[[1480   20]
 [  42 1457]]
COVID-19 [0.94308943 0.05691057] 2.8076800420510515
```




GLCM_correlation_135
GLCM_homogeneity_90
GLCM_contrast_90
W_7
GLCM_ASM_0
GLCM_homogeneity_45
GLCM_energy_90
GLCM_correlation_0
GLCM_homogeneity_0
W_5
0.0
0.1
0.2
0.3
0.4


GLCM_correlation_135 <= 0.364
gini = 0.5
samples = 11993
value = [5996, 5997]
class = Non-COVID-19
GLCM_contrast_90 <= 1.653
gini = 0.379
samples = 7710
value = [5752, 1958]
class = COVID-19
GLCM_energy_90 <= -0.705
gini = 0.107
samples = 4283
value = [244, 4039]
class = Non-COVID-19
GLCM_ASM_0 <= -0.248
gini = 0.33
samples = 7004
value = [5545, 1459]
class = COVID-19
GLCM_homogeneity_45 <= -0.555
gini = 0.414
samples = 706
value = [207, 499]
class = Non-COVID-19
W_7 <= -2.114
gini = 0.497
samples = 460
value = [212, 248]
class = Non-COVID-19
GLCM_correlation_45 <= 0.254
gini = 0.017
samples = 3823
value = [32, 3791]
class = Non-COVID-19
GLCM_homogeneity_90 <= -0.603
gini = 0.407
samples = 5006
value = [3568, 1423]
class = COVID-19
GLCM_contrast_0 <= -0.953
gini = 0.035
samples = 1998
value = [1962, 36]
class = COVID-19
GLCM_energy_0 <= -0.901
gini = 0.383
samples = 279
value = [207, 72]
class = COVID-19
gini = 0.0
samples = 427
value = [0, 427]
class = Non-COVID-19
gini = 0.0
samples = 172
value = [0, 172]
class = Non-COVID-19
GLCM_correlation_90 <= 0.828
gini = 0.389
samples = 288
value = [212, 76]
class = COVID-19
gini = 0.0
samples = 5
value = [5, 0]
class = COVID-19
GLCM_correlation_45 <= 0.285
gini = 0.014
samples = 3818
value = [27, 3791]
class = Non-COVID-19